

Aim: Data Analysis: Getting to know the Data (Using R Programming)

- Parametric: Means, T-Test, Correlation
- Prediction for numerical outcomes – Linear regression, Multiple Linear Regression
- Correlation analysis
- Preparing data for analysis

1.Loading the dataset.

```
import pandas as pd
import numpy as np
from scipy import stats
import statsmodels.api as sm
import statsmodels.formula.api as smf
import matplotlib.pyplot as plt
```

Matplotlib is building the font cache; this may take a moment.

```
import micropip
await micropip.install(['seaborn'])
```

```
from pyodide.http import open_url
import pandas as pd
url = 'https://raw.githubusercontent.com/mwaskom/seaborn-data/master/mpg.csv'
with open_url(url) as f:
    df = pd.read_csv(f)
```

2.Viewing the structure.

```
df.head()
```

	mpg	cylinders	displacement	horsepower	weight	acceleration	model_year	origin	name
0	18.0	8	307.0	130.0	3504	12.0	70	usa	chevrolet chevelle malibu
1	15.0	8	350.0	165.0	3693	11.5	70	usa	buick skylark 320
2	18.0	8	318.0	150.0	3436	11.0	70	usa	plymouth satellite
3	16.0	8	304.0	150.0	3433	12.0	70	usa	amc rebel sst
4	17.0	8	302.0	140.0	3449	10.5	70	usa	ford torino

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 398 entries, 0 to 397
Data columns (total 9 columns):
#   Column          Non-Null Count  Dtype
---  -
0   mpg              398 non-null    float64
1   cylinders         398 non-null    int64
2   displacement      398 non-null    float64
3   horsepower        392 non-null    float64
4   weight            398 non-null    int64
5   acceleration      398 non-null    float64
6   model_year        398 non-null    int64
7   origin            398 non-null    object
8   name              398 non-null    object
dtypes: float64(4), int64(3), object(2)
memory usage: 24.9+ KB
```

```
df.describe()
```

	mpg	cylinders	displacement	horsepower	weight	acceleration	model_year
count	398.000000	398.000000	398.000000	392.000000	398.000000	398.000000	398.000000
mean	23.514573	5.454774	193.425879	104.469388	2970.424623	15.568090	76.010050
std	7.815984	1.701004	104.269838	38.491160	846.841774	2.757689	3.697627
min	9.000000	3.000000	68.000000	46.000000	1613.000000	8.000000	70.000000
25%	17.500000	4.000000	104.250000	75.000000	2223.750000	13.825000	73.000000
50%	23.000000	4.000000	148.500000	93.500000	2803.500000	15.500000	76.000000
75%	29.000000	8.000000	262.000000	126.000000	3608.000000	17.175000	79.000000
max	46.600000	8.000000	455.000000	230.000000	5140.000000	24.800000	82.000000

3.Check missing values.

```
print(df.isnull().sum())
```

```
mpg          0
cylinders     0
displacement  0
horsepower    6
weight        0
acceleration  0
model_year    0
origin        0
name          0
dtype: int64
```

4.Dropping the missing values, converting 'cylinders' to categorical and checking the data types and info.

```
df_clean = df.dropna().copy()
```

```
df_clean['cylinders'] = df_clean['cylinders'].astype('category')
```

```
print(df_clean.info())
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 392 entries, 0 to 397
Data columns (total 9 columns):
#   Column          Non-Null Count  Dtype
---  -
0   mpg             392 non-null   float64
1   cylinders       392 non-null   category
2   displacement    392 non-null   float64
3   horsepower      392 non-null   float64
4   weight          392 non-null   int64
5   acceleration    392 non-null   float64
6   model_year      392 non-null   int64
7   origin          392 non-null   object
8   name            392 non-null   object
dtypes: category(1), float64(4), int64(2), object(2)
memory usage: 25.0+ KB
None
```

5. Printing overall mpg stats.

```
print(df_clean['mpg'].describe())
```

```
count    392.000000
mean      23.445918
std        7.805007
min        9.000000
25%       17.000000
50%       22.750000
75%       29.000000
max       46.600000
Name: mpg, dtype: float64
```

6. Printing mean of mpg by cylinder with matrix.

```
print(df_clean.groupby('cylinders')['mpg'].agg(['mean', 'std', 'count']))
```

```
<ipython-input-19-ae908608bbf0>:1: FutureWarning: The default of observed
s. Pass observed=False to retain current behavior or observed=True to add
print(df_clean.groupby('cylinders')['mpg'].agg(['mean', 'std', 'count']))
```

	mean	std	count
cylinders			
3	20.550000	2.564501	4
4	29.283920	5.670546	199
5	27.366667	8.228204	3
6	19.973494	3.828809	83
8	14.963107	2.836284	103

7.T-Test and comparing MPG for 4 vs 6 cylinders.

```
group_4 = df_clean[df_clean['cylinders'] == 4]['mpg']
group_6 = df_clean[df_clean['cylinders'] == 6]['mpg']
```

```
t_stat,p_val = stats.ttest_ind(group_4,group_6,equal_var=False)
print(f"T-statistic: {t_stat:.3f},P-value:{p_val:.3e}")
```

T-statistic: 16.010,P-value:6.245e-39

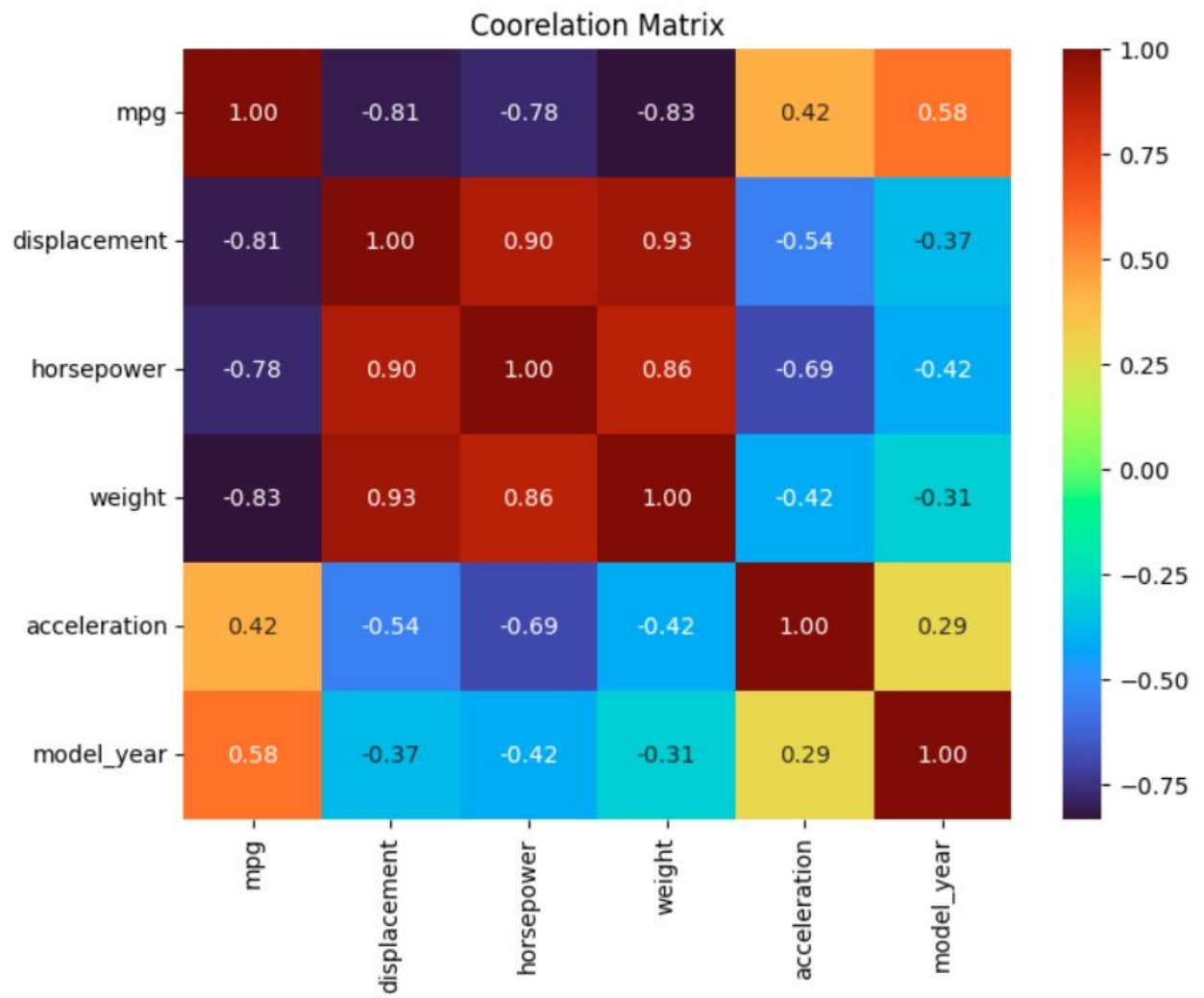
8.Selecting numeric columns for correlation matrix and printing it.

```
num_cols = df_clean.select_dtypes(include=np.number)
corr_matrix = num_cols.corr()
print(corr_matrix)
```

	mpg	displacement	horsepower	weight	acceleration	\
mpg	1.000000	-0.805127	-0.778427	-0.832244	0.423329	
displacement	-0.805127	1.000000	0.897257	0.932994	-0.543800	
horsepower	-0.778427	0.897257	1.000000	0.864538	-0.689196	
weight	-0.832244	0.932994	0.864538	1.000000	-0.416839	
acceleration	0.423329	-0.543800	-0.689196	-0.416839	1.000000	
model_year	0.580541	-0.369855	-0.416361	-0.309120	0.290316	

	model_year
mpg	0.580541
displacement	-0.369855
horsepower	-0.416361
weight	-0.309120
acceleration	0.290316
model_year	1.000000

```
import seaborn as sns
plt.figure(figsize=(8,6))
sns.heatmap(corr_matrix,annot=True,cmap='turbo',fmt=".2f")
plt.title('Coorelation Matrix')
plt.show()
```



1.Creating a linear regression (predict mpg and horsepower) .

```
model1= smf.ols('mpg~horsepower',data=df_clean).fit()
print(model1.summary())
```

OLS Regression Results

Dep. Variable:	mpg	R-squared:	0.606
Model:	OLS	Adj. R-squared:	0.605
Method:	Least Squares	F-statistic:	599.7
Date:	Fri, 04 Jul 2025	Prob (F-statistic):	7.03e-81
Time:	09:29:34	Log-Likelihood:	-1178.7
No. Observations:	392	AIC:	2361.
Df Residuals:	390	BIC:	2369.
Df Model:	1		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	39.9359	0.717	55.660	0.000	38.525	41.347
horsepower	-0.1578	0.006	-24.489	0.000	-0.171	-0.145

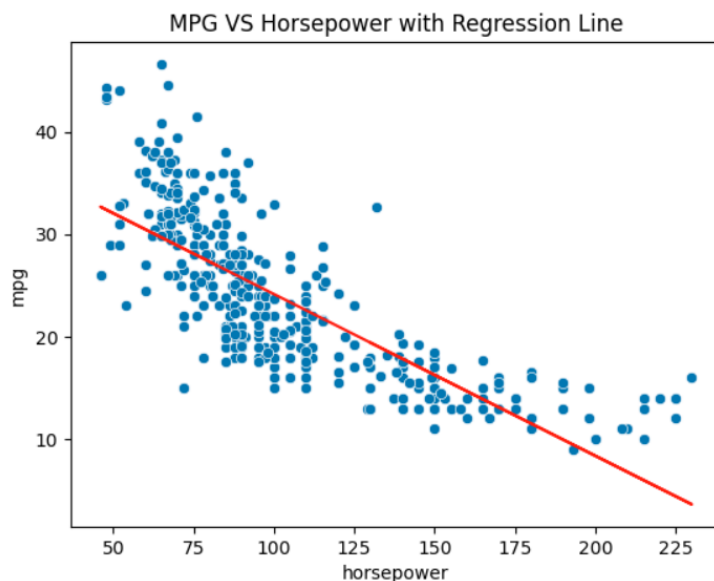
Omnibus:	16.432	Durbin-Watson:	0.920
Prob(Omnibus):	0.000	Jarque-Bera (JB):	17.305
Skew:	0.492	Prob(JB):	0.000175
Kurtosis:	3.299	Cond. No.	322.

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

2.Plotting the linear regression line.

```
sns.scatterplot(x='horsepower',y='mpg',data=df_clean)
plt.plot(df_clean['horsepower'],model1.fittedvalues,color='red')
plt.title('MPG VS Horsepower with Regression Line')
plt.show()
```



3.Creating multiple Linear Regression (mpg~Horsepower+weight+acceleration) and summary of it.

```
model2=smf.ols('mpg~horsepower+weight+acceleration',data=df_clean).fit()
print(model2.summary())
```

```

=====
                        OLS Regression Results
=====
Dep. Variable:          mpg      R-squared:                0.706
Model:                  OLS      Adj. R-squared:            0.704
Method:                 Least Squares      F-statistic:          311.1
Date:                   Fri, 04 Jul 2025    Prob (F-statistic):      7.48e-103
Time:                   09:43:01    Log-Likelihood:         -1121.0
No. Observations:       392      AIC:                   2250.
Df Residuals:           388      BIC:                   2266.
Df Model:                3
Covariance Type:        nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
Intercept	45.6783	2.409	18.965	0.000	40.943	50.414
horsepower	-0.0475	0.016	-2.970	0.003	-0.079	-0.016
weight	-0.0058	0.001	-10.024	0.000	-0.007	-0.005
acceleration	-0.0021	0.123	-0.017	0.987	-0.245	0.240

```

=====
Omnibus:                 35.392    Durbin-Watson:           0.858
Prob(Omnibus):            0.000    Jarque-Bera (JB):         46.071
Skew:                     0.684    Prob(JB):                 9.91e-11
Kurtosis:                 3.975    Cond. No.                 3.48e+04
=====

```

Notes:

- [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
- [2] The condition number is large, 3.48e+04. This might indicate that there are strong multicollinearity or other numerical problems.